Sprawozdanie

Zadanie 2

Mikołaj Baliszewski 319261

Numery lotów:

LO527, X37106

Celem ćwiczenia była transformacja współrzędnych elipsoidalnych lecącego samolotu, do układu lokalnego względem położenia lotniska wylotu. Danymi do zadania są dane lotu pobrane z portalu *flightradar24.com*. Na podstawie danych i obliczeń wykonane zostały wizualizacje. Zadanie wykonano w języku programowania Python w następujących krokach:

1. Zaimportowanie potrzebnych bibliotek i plików:

```python
import matplotlib.pyplot as plt
import cartopy.crs as ccrs
import cartopy.io.img_tiles as cimgt
import numpy as np
from read_flightradar import read_flightradar
```

Wykorzystywane biblioteki to *cartopy*, *matplotlib oraz numpy*, zaś plik *read_flightradar.py* dostarczony został na zajęciach i służy do odczytu danych lotu w formacie *.csv* i zapis tych danych do tablicy.

2. Utworzenie klasy *flight*:

```python
class flight:
    def __init__(self, flight_data_dir):
        self.data = read_flightradar(flight_data_dir)   #odczytanie danych
        self.date = np.column_stack((self.data[:, 1], self.data[:, 2], self.data[:, 3]))    #data
        self.hms = np.column_stack((self.data[:, 4], self.data[:, 5], self.data[:, 6]))     #godzina
        self.speed = self.data[:, -2] * 1.852       #prędkość w węzłach przeliczona na km/h
        self.lat = np.deg2rad(self.data[1:, -5])    #szerokość geograficzna samolotu
        self.lon = np.deg2rad(self.data[1:, -4])    #długość geograficzna samolotu
        self.h = self.data[1:, -3] * 0.3048         #h w stopach * 0.3048 = h w metrach, wysokość względem lotniska
        self.lat_l = np.deg2rad(self.data[0, -5])   #szerokość geograficzna lotniska
        self.lon_l = np.deg2rad(self.data[0, -4])   #długość geograficzna lotniska
        self.h_l = self.data[0, -3]                 #wysokość lotniska = 0

        self.X, self.Y, self.Z = self.XYZ(self.lat, self.lon, self.h)             #XYZ samolotu
        self.X_l, self.Y_l, self.Z_l = self.XYZ(self.lat_l, self.lon_l, self.h_l) #XYZ lotniska
        self.vec()      #utworzenie wektora samolot - lotnisko we wsp. ortokartezjańskich
        self.to_neu()   #współrzędne samolotu w układzie lokalnym
        self.s, self.az, self.h_h = self.s_az_h(self.neu) #obliczenie odległości, azymutu, elewacji względem lotniska
```

W klasie znajduje się jedno parametrowy konstruktor do którego przekazujemy ścieżkę do danych lotu. Za pomocą funkcji *read_flightradar()* dane zostają odczytane i zapisane w tablicy *data*, następnie utworzone zostają tablice *date, hms, speed, lat, lon, h, lat_l, lon_l, h_l*, które przechowują wybrane dane lotu (na dołączonym zrzucie ekranu widać, które dokładnie dane przechowywane są w poszczególnych tablicach). Następnie wywołane zostają funkcje obliczające: współrzędne geocentryczne samolotu i lotniska (*XYZ()*), wektor samolot – lotnisko (*vec()*), współrzędne samolotu w układzie lotniska początkowego (*to_neu()*) oraz odległość, azymut i wysokość względem lotniska początkowego (*s_az_h()*). Definicje tych funkcji podane są w następnych punktach.

3. Przeliczenie współrzędnych geograficznych do geocentrycznych współrzędnych ortokartezjańskich.

```python
    #obliczenie geocentrcznych współrzędnych ortokartezjańskich
    def XYZ(self, phi, lam, h):
        a = 6378137             #wielka półoś dla elipsoidy GRS80
        e = 0.00669438002290    #kwadrat pierwszego mimośrodu dla elipsoidy GRS80
        n = a / (np.sqrt(1 - e*np.power(np.sin(phi), 2))) #promień przekroju Ziemi w kierunku I Wertykału
        X = (n + h)*np.cos(phi)*np.cos(lam)
        Y = (n + h)*np.cos(phi)*np.sin(lam)
        Z = (n * (1-e) + h)*np.sin(phi)
        return [X, Y ,Z]
```

Argumentami funkcji są współrzędne geodezyjne, do których zastosowano następujące wzory:

Do obliczenia *n:*

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}$$

Do obliczenia współrzędnych ortokartezjańskich X, Y, Z:

$$X = (N + h) \cos \varphi \cos \lambda$$
$$Y = (N + h) \cos \varphi \sin \lambda$$
$$Z = [N(1 - e^2) + h] \sin \varphi$$

4. Obliczenie wektora samolot – lotnisko we współrzędnych ortokartezjańskich.

```
def vec(self):
    XYZ = np.column_stack((self.X, self.Y, self.Z))          #tablica z wsp. XYZ samolotu
    XYZ_l = np.column_stack((self.X_l, self.Y_l, self.Z_l)) #tablica z wsp. XYZ lotniska
    self.vector_XYZ = np.subtract(XYZ, XYZ_l)                #odejmowanie wsp. lotniska od wsp. samolotu
```

W celu obliczenia wektora od współrzędnych samolotu odjęte zostały współrzędne lotniska.

5. Przeliczenie współrzędnych geocentrycznych do układu lokalnego n, e, u względem lotniska początkowego.

```
def to_neu(self):
    #macierz obrotu
    R_matrix = np.array([[-np.sin(self.lat_l) * np.cos(self.lon_l), -np.sin(self.lon_l), np.cos(self.lat_l) * np.cos(self.lon_l)],
                         [-np.sin(self.lat_l) * np.sin(self.lon_l), np.cos(self.lon_l), np.cos(self.lat_l) * np.sin(self.lon_l)],
                         [np.cos(self.lat_l), 0, np.sin(self.lat_l)]])
    #transponacja macierzy
    R_matrix = np.transpose(R_matrix)

    #mnożenie macierzy przez wektor samolot-lotnisko i uzyskanie wsp. neu
    self.neu = np.empty([len(self.lon), 3])
    for i in range(len(self.lon)):
        self.neu[i] = np.matmul(R_matrix, self.vector_XYZ[i])
```

W tym celu utworzona została następująca macierz obrotu między układem współrzędnych geocentrycznych i lokalnych:

$$R_{\text{neu}} = \left[\vec{n}\ \vec{e}\ \vec{u}\right] = \begin{bmatrix} -\sin\varphi\cos\lambda & -\sin\lambda & \cos\varphi\cos\lambda \\ -\sin\varphi\sin\lambda & \cos\lambda & \cos\varphi\sin\lambda \\ \cos\varphi & 0 & \sin\varphi \end{bmatrix}$$

Następnie macierz ta została przetransponowana, a wektor współrzędnych n, e, u, zapisany w tablicy *neu*, obliczony został poprzez pomnożenie macierzy obrotu przez wektor samolot – lotnisko we współrzędnych ortokartezjańskich.

6. Obliczenie odległości od lotniska początkowego oraz azymutu i wysokości w układzie lotniska początkowego.

```
def s_az_h(self, neu):
    #współrzędne neu, w układzie lotniska wylotu
    n = neu[:, 0]
    e = neu[:, 1]
    u = neu[:, 2]

    #odległość, azymut, wysokość
    s = np.sqrt(np.power(n, 2)+np.power(e, 2)+np.power(u, 2))
    az = np.arctan2(e, n)
    h = np.arcsin(u/s)
    return [s, az, h]
```

Odległość, azymut i wysokość obliczone zostały na podstawie następujących wzorów:

$$s = \sqrt{n^2 + e^2 + u^2} \quad Az = \arctan\frac{e}{n} \quad h = \arcsin\left(\frac{u}{\sqrt{n^2 + e^2 + u^2}}\right)$$

7. Utworzenie funkcji tworzącej wizualizację trasy samolotu na podkładzie mapy.

```python
#wykres trasy na podkładzie mapy
def map_plot(self):
    fig = plt.figure(figsize=(9, 9))

    osm_tiles = cimgt.OSM()  # mapa OpenStreetMap
    extent = [np.rad2deg(min(self.lon)) - 5, np.rad2deg(max(self.lon)) + 5,
              np.rad2deg(min(self.lat)) - 5, np.rad2deg(max(self.lat)) + 5]    #wyznaczenie przybliżenia
    ax = plt.axes(projection=osm_tiles.crs)
    ax.set_extent(extent, crs=ccrs.Geodetic())
    if max(self.s/1000) < 550:  #dokładność mapy, dla dłuższego lotu mniejsze przybliżenie, więc mniejsza dokładność
        ax.add_image(osm_tiles, 5)
    else:
        ax.add_image(osm_tiles, 4)

    #ponowne złączenie współrzędnych lotniska i samolotu
    lat = np.insert(np.rad2deg(self.lat), 0, np.rad2deg(self.lat_l))
    lon = np.insert(np.rad2deg(self.lon), 0, np.rad2deg(self.lon_l))

    #punkt w którym samolot znika z horyzontu
    lat_under, lon_under = self.point_under()

    #rysowanie
    ax.plot(lon, lat, transform=ccrs.PlateCarree(), label='Trasa samolotu')  # rzeczywista trasa samolotu
    ax.scatter(lon_under, lat_under, c='red', s=20,  transform=ccrs.PlateCarree(), label='Punkt zniknięcia za horyzontem')
    ax.legend(fontsize=12)

    #zapis
    fig.savefig(str(np.round(np.rad2deg(self.lon_l), 2)) + '_map_plot.png', format='png', pad_inches=0.1,
                dpi='figure')
```

Jako podkład mapy użyta została mapa OpenStreetMap, w zmiennej *extent*, zawarte są minimalne i maksymalne współrzędne danego kawałka mapy (dopasowane do trasy samolotu). Oprócz przedstawienia trasy samolotu prezentowany jest również punkt od, którego samolot znajduje się pod płaszczyzną horyzontu. Do obliczenia współrzędnych tego punktu wykorzystana została funkcja *point_under()*, której definicja jest następująca:

```python
#obliczenie wspołrzędnych geograficznych pierwszego punktu w którym samolot jest pod horyzontem
def point_under(self):
    lat_under = 0
    lon_under = 0
    for i in range(len(self.h)):
        if self.h[i] > 0 and self.h_h[i] < 0:
            lat_under = self.lat[i]
            lon_under = self. lon[i]
            break
    return [np.rad2deg(lat_under), np.rad2deg(lon_under)]
```

8. Utworzenie funkcji tworzącej wykres typu sky plot.

```python
#wykres sky plot w układzie lotniska początkowego do momentu zniknięcia samolotu za horyzontem
def sky_plot(self):
    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(polar=True)
    ax.set_theta_zero_location('N')  # ustawienie kierunku północy na górze wykresu
    ax.set_theta_direction(-1)
    ax.set_title('Położenie samolotu w układzie lotniska początkowego', size = 17)

    #wybranie danych dla których samolot znajduje się nad horyzontem
    az_cpy = []
    s_cpy = []
    for i in range(len(self.h)):
        if self.h[i] > 0 and self.h_h[i] > 0:
            az_cpy.append(self.az[i])
            s_cpy.append(self.s[i]/1000)

    # narysowanie punktu na wykresie
    ax.scatter(az_cpy, s_cpy, color='blue', s=4)

    #zapis do pliku
    fig.savefig(str(np.round(np.rad2deg(self.lon_l), 2)) + '_sky_plot.png', format='png', pad_inches=0.1,
                dpi='figure')
```

Na wykresie przedstawione jest położenie samolotu względem lotniska początkowego (wybrane zostały, tylko te dane dla, których samolot znajduje się nad horyzontem). Na osi x przedstawiony jest azymut, zaś na osi y odległość od lotniska początkowego w kilometrach.

9. Utworzenie funkcji tworzącej wykres przedstawiający wysokość samolotu względem lotniska wylotu w zależności od godziny lotu.

```python
#wykres wysokości samolotu względem lotniska wylotu
def height_plot(self):
    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot()
    ax.set_title("Wysokość samolotu(względem lotniska początkowego) w zależności od czasu", size=12)
    ax.grid()
    ax.set_xlabel('Godzina', size = 18)
    ax.set_ylabel('Wysokość (m)', size=18)

    time, ticks, ticks_labels = self.time_labels()          #podziałka i opisy osi x

    ax.set_yticks(range(0, int(max(self.h)) + 500, 500))    #podziałka na osi y
    ax.set_xticks(ticks)
    ax.set_xticklabels(ticks_labels)
    h = np.insert(self.h, 0, self.h_l)                      #złączenie wartości wysokości samolotu i lotniska

    #rysowanie
    ax.plot(time, h, linewidth=1)

    #zapis
    fig.savefig(str(np.round(np.rad2deg(self.lon_l), 2)) + '_height.png', format='png', pad_inches=0.1,
                dpi='figure')
```

Na osi x zaprezentowana została godzina lotu, zaś na osi y odległość samolotu od lotniska początkowego w metrach. Do utworzenia wartości, podziałki i opisów wartości na osi x użyta została funkcja *time_labels()*, której definicja jest następująca:

```python
    #wybranie wartości i opisów na osi x dla wykresów z godziną lotu na osi x
    def time_labels(self):
        time = np.empty(len(self.hms))  #tablica z czasem przeliczonym z hms na godziny
        ticks = []                      #wartośći przedsatwione na osi x
        ticks_labels = []               #opisy wartości przedstawianych na osi x
        if max(self.s/1000) < 550:      #odstęp czasu między wartościami (w godzinach)
            time_interval = 0.2
        else:
            time_interval = 0.8
        prev = 0
        for i in range(len(self.hms)):
            time[i] = (float(self.hms [i][0]) + float(self.hms[i][1])/60 + float(self.hms[i][2])/3600)
            #wybranie wartości dla ustawionego interwału czasu, momentu startu i lądowania
            if time[i]-prev > time_interval or i==0 or i==len(self.hms)-1:
                ticks.append(time[i])
                label = str(int(self.hms[i][0])) + ':' + str(int(self.hms[i][1]))
                if label[2] == ':' and len(label) != 5:
                    label = label[:2] + ':0' + label[3]
                if label[1] == ':' and len(label) == 3:
                    label = '0' + label[0] + ':0' + label[2]
                if label[1] == ':':
                    label = '0' + label
                ticks_labels.append(label)
                prev = time[i]
        return [time, ticks, ticks_labels]
```

10. Utworzenie funkcji tworzącej wykres przedstawiający prędkość samolotu w zależności od godziny lotu.

```python
    #wykres prędkości samolotu w zależności od godziny lotu
    def speed_plot(self):
        fig = plt.figure(figsize=(8, 8))
        ax = fig.add_subplot()
        ax.set_title("Prędkość samolotu w zależności od czasu", size=15)
        ax.grid()
        ax.set_xlabel('Godzina', size = 18)
        ax.set_ylabel('Prędkość (km/h)', size=18)

        time, ticks, ticks_labels = self.time_labels()          #podziałka i opisy osi x

        ax.set_yticks(range(0, int(max(self.speed)) + 100, 50)) #podziałka na osi y
        ax.set_xticks(ticks)
        ax.set_xticklabels(ticks_labels)

        #rysowanie
        ax.plot(time, self.speed, linewidth=1)

        #zapis
        fig.savefig(str(np.round(np.rad2deg(self.lon_l), 2)) + '_speed.png', format='png', pad_inches=0.1,
                    dpi='figure')
```

Na osi x znajduje się godzina lotu, zaś na osi y prędkość samolotu w kilometrach na godzinę. Ponownie zastosowana została funkcja *time_labels()*, zaś podziałka no osi y ustawiona została co 50 jednostek.

11. Utworzenie funkcji przedstawiającej odległość samolotu od lotniska wylotu w zależności od godziny lotu.

```python
#wykres odległości samolotu od lotniska wylotu w zależności od godziny
def distance_plot(self):
    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot()
    ax.set_title("Odległość od lotniska w zależności od czasu", size=15)
    ax.grid()
    ax.set_xlabel('Godzina', size = 18)
    ax.set_ylabel('Odległość (km)', size=18)

    time, ticks, ticks_labels = self.time_labels() #podziałka i opisy osi x
    s = np.insert(self.s, 0, 0) #dodanie do tablicy z odległościami, odległości równej 0 jako pierwszy element

    #punkt w którym samolot znika z horyzontu
    lat_under, lon_under = self.point_under()
    for i in range(len(self.lat)):
        if self.lat[i] == np.deg2rad(lat_under):
            lon_under = time[i]
            lat_under = s[i]/1000
            break

    #ustawienie podziałki na osi y w zależności od długości lotu
    for i in s:
        if (i > 550000):
            ax.set_yticks(range(0, int(max(self.s)/1000) + 200, 200))
            break
        else:
            ax.set_yticks(range(0, int(max(self.s)/1000) + 50, 25))

    ax.set_xticks(ticks)
    ax.set_xticklabels(ticks_labels)

    #rysowanie
    ax.plot(time, s/1000, linewidth=1)  #ogległość
    ax.scatter(lon_under, lat_under, c='red', s=20, label='Punkt zniknięcia za horyzontem') #punkt zniknięcia
    ax.legend()

    #zapis
    fig.savefig(str(np.round(np.rad2deg(self.lon_l), 2)) + '_distance.png', format='png', pad_inches=0.1,
                dpi='figure')
```

Na osi x przedstawiono godzinę lotu, ponownie użyto funkcji *time_labels()*. Na osi y umieszczono odległość od lotniska w kilometrach. Podziałkę na osi y ustawiono co 25 lub co 200 kilometrów, w zależności od długości lotu. Na wykresie zaprezentowano również punkt od, którego samolot znajduje się poniżej płaszczyzny horyzontu. W tym celu użyto funkcji *point_under()* i dla jej wyniku wyznaczono odpowiadającą, współrzędnym geograficznym samolotu, godzinę lotu i odległość od lotniska.

12. Utworzenie funkcji tworzącej wykres przedstawiający kąt elewacji względem lotniska początkowego w zależności od odległości od tego lotniska.

```python
    #wykres wartości wysokości horyzontalnej względem lotniska początkowego w zależności od odległości
    def horizontal_height_plot(self):
        fig = plt.figure(figsize=(8, 8))
        ax = fig.add_subplot()
        ax.set_title("Kąt elewacji w zależności od odległości od lotniska wylotu", size=15)
        ax.grid()
        ax.set_xlabel('Odległość (km)', size = 18)
        ax.set_ylabel('Wysokość (°)', size=18)

        # ustawienie podziałki na osi x w zależności od długości lotu
        for i in self.s:
            if (i > 550000):
                ax.set_xticks(range(0, int(max(self.s)/1000), 500))
        ax.set_yticks(range(-90, 90, 5))    #ustawienie podziałki na osi x

        #rysowanie
        ax.plot(self.s/1000, np.rad2deg(self.h_h), linewidth=1)

        #zapis
        fig.savefig(str(np.round(np.rad2deg(self.lon_l), 2)) + '_horizontal_height.png', format='png', pad_inches=0.1,
                    dpi='figure')
```

Na osi x zaprezentowano odległość od lotniska w kilometrach, zaś na osi y wysokość względem lotniska w stopniach. Na osi x, zwiększono wartość podziałki dla dłuższych lotów, a działka podziałki na osi y wynosi 5 stopni.

13. Utworzenie 2 obiektów klasy flight i wywołanie dla każdego z nich funkcji tworzących wizualizacje.

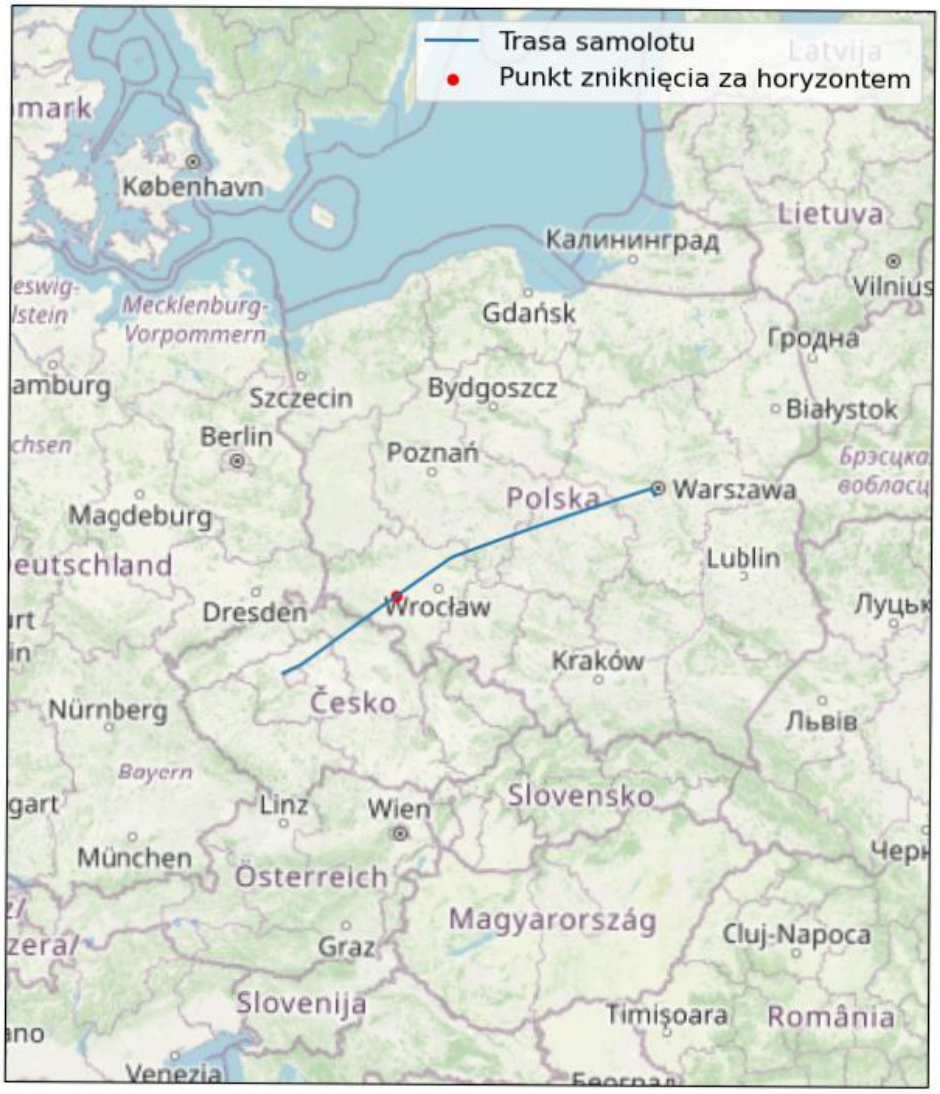
```python
lot527 = flight('E:\sem3\geodezja\z2\LO527_2deffcc6.csv')
lot527.map_plot()
lot527.sky_plot()
lot527.height_plot()
lot527.speed_plot()
lot527.distance_plot()
lot527.horizontal_height_plot()

X37106 = flight('E:\sem3\geodezja\z2\_boa_vista.csv')
X37106.map_plot()
X37106.sky_plot()
X37106.height_plot()
X37106.speed_plot()
X37106.distance_plot()
X37106.horizontal_height_plot()
```

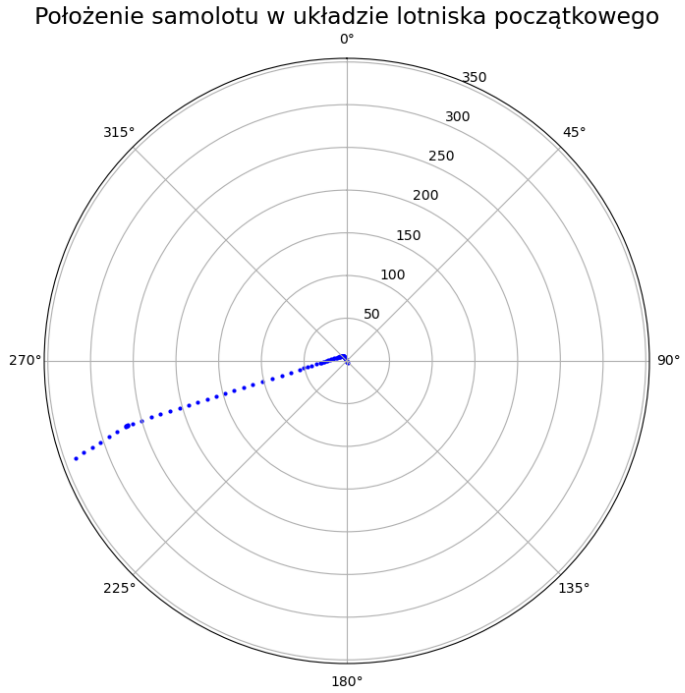
Obiekt *lot527* klasy flight dotyczy lotu o numerze LO527, jest to lot z Warszawy z Lotniska Chopina do Pragi na lotnisko Václava Havla dnia 22.10.2022, zaś obiekt *X37106* jest dodatkowym lotem (nie podanym w danych do zadania) i dotyczy lotu o numerze X37106 z wyspy Sal (Republika Zielonego Przylądka) z lotniska Amilcara Cabrala do lotniska we Frankfurcie dnia 06.11.2022.

Wizualizacje LO527:

1. Trasa samolotu na podkładzie mapy.

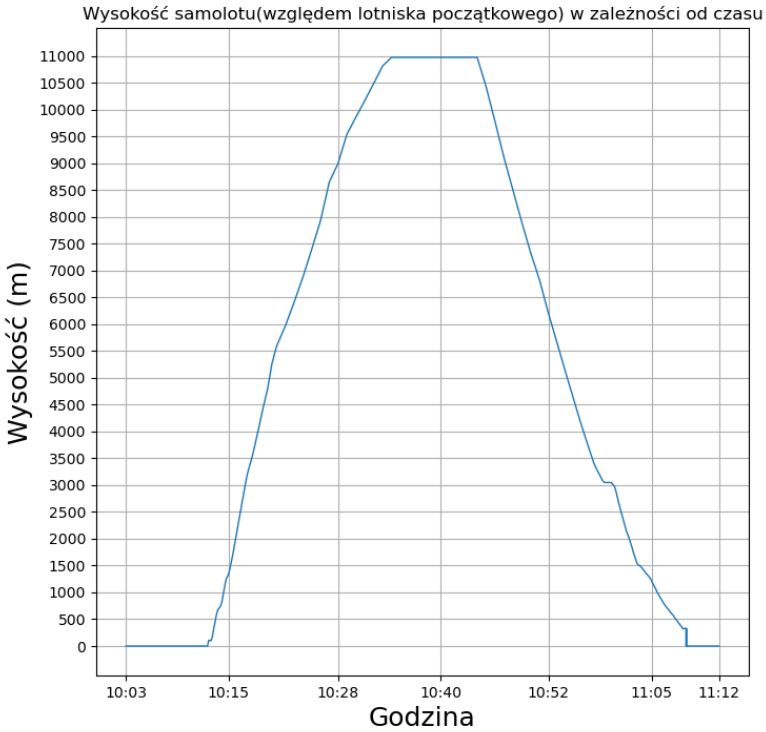


2. Wykres sky plot w układzie lotniska początkowego.

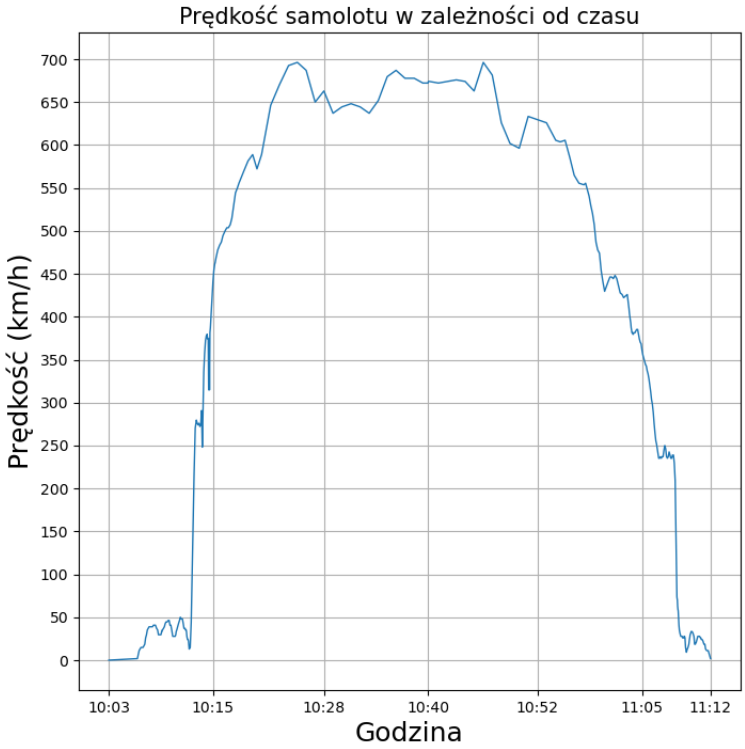


Samolot znika za horyzontem między 300, a 350 kilometrem lotu. Do tego czasu jego azymut względem lotniska początkowego ma wartość między 225, a 270 stopni.

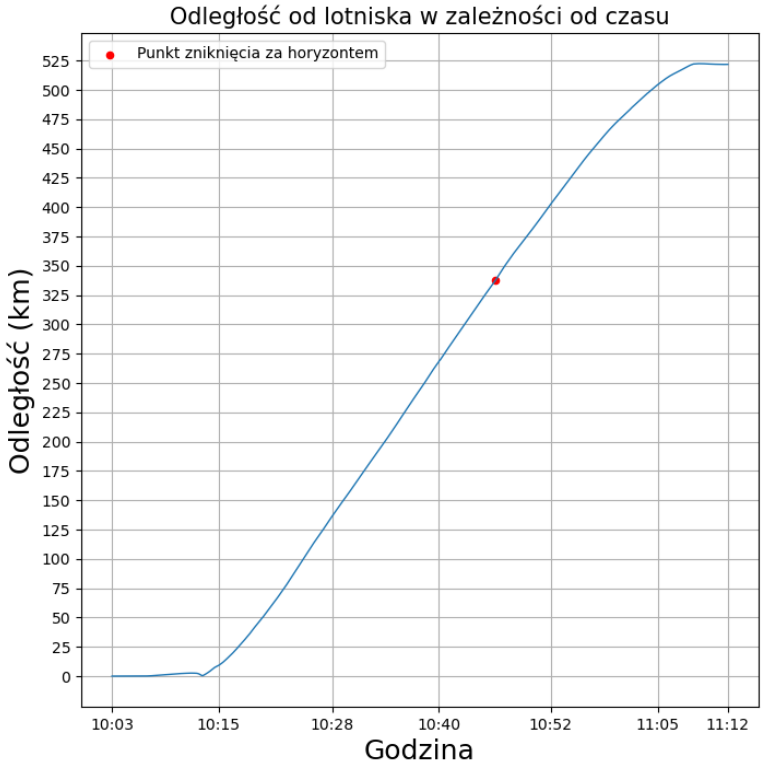
3. Wykres wysokości względem lotniska początkowego.



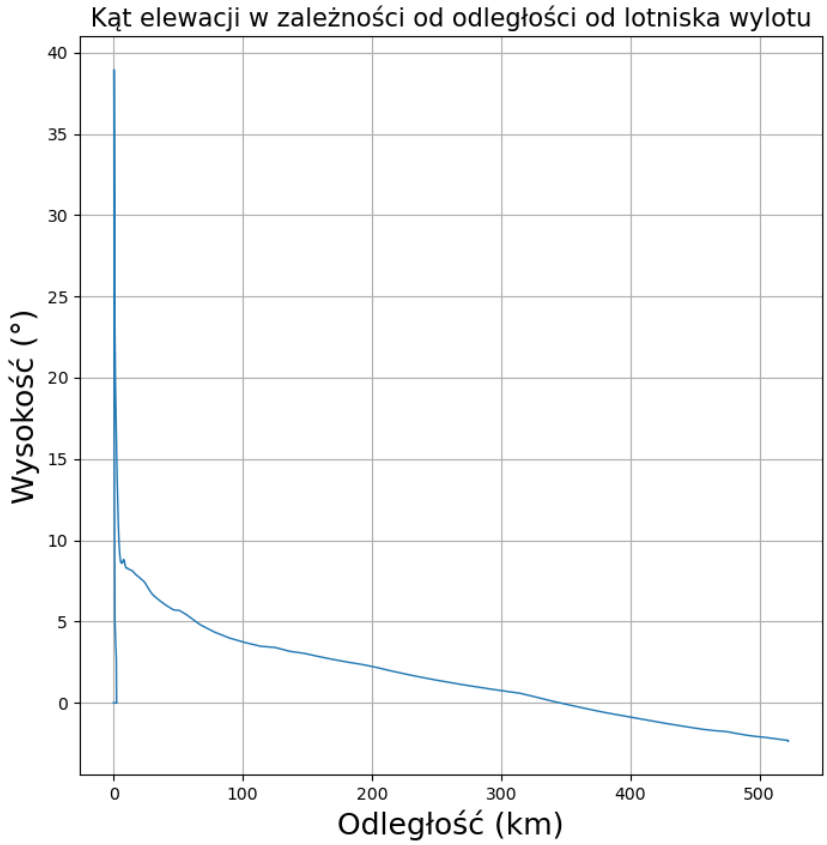
4. Wykres prędkości samolotu.



5. Wykres odległości od lotniska początkowego.



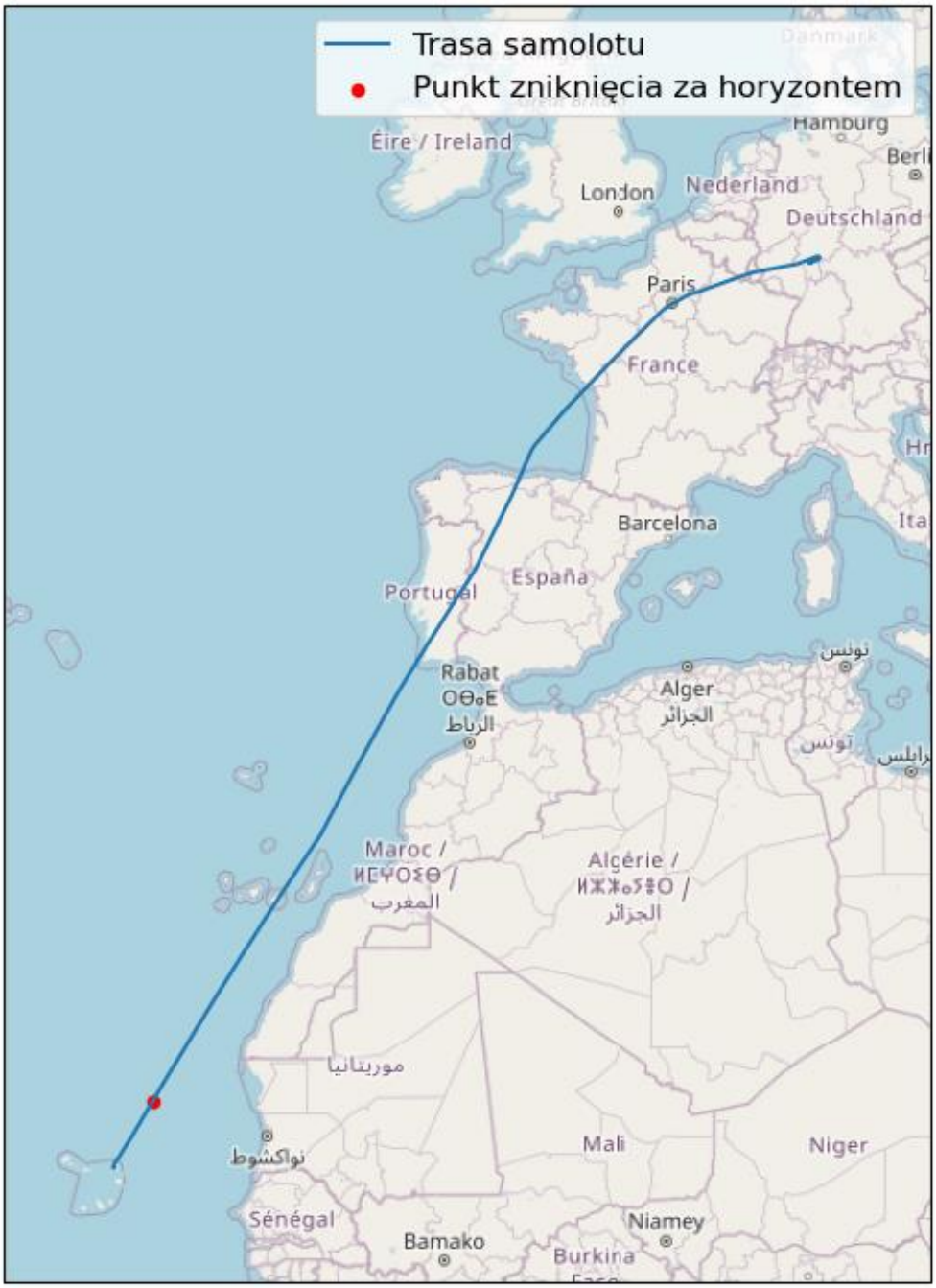
6. Wykres kąta elewacji względem lotniska początkowego.



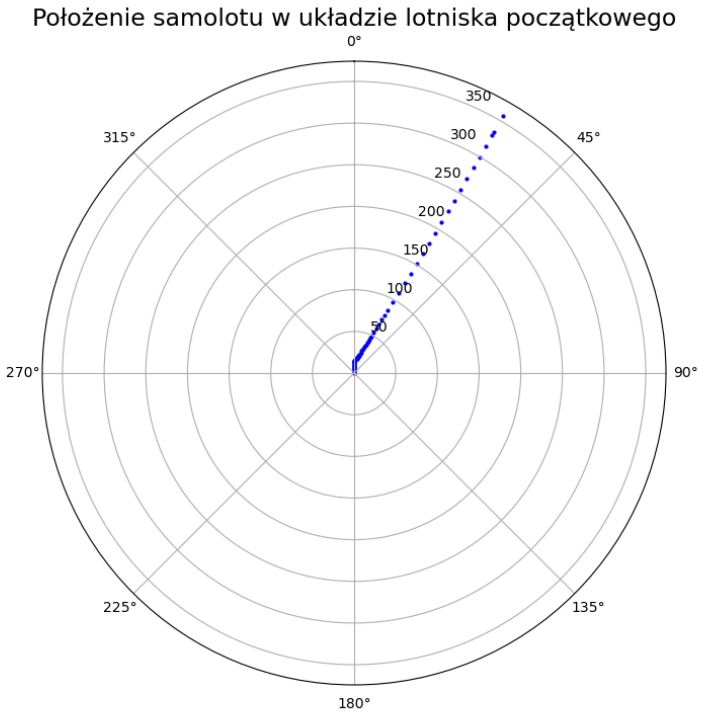
Z wizualizacji wynika, że lot trwał godzinę i 9 minut, samolot przez ponad połowę lotu znajdował się nad horyzontem, pod płaszczyzną horyzontu znalazł się na około 340 kilometrze lotu około godziny 10:46. Cała długość trasy jaką pokonał to około 525 kilometrów. Przez prawie połowę lotu wznosił się, wysokość przelotową (11000m) osiągnął tylko na niecałe 10 minut, następnie przez resztę trasy obniżał lot. Maksymalna prędkość wynosiła około 700km/h, wraz z rozpoczęciem obniżania lotu prędkość również malała. Maksymalny kąt elewacji wyniósł około 38 stopni, wraz z oddalaniem się od lotniska wartość kąta elewacji gwałtownie spadła, a na lotnisku w Pradze wyniosła około -3 stopnie.

Wizualizacje X37106:

1. Trasa samolotu na podkładzie mapy.

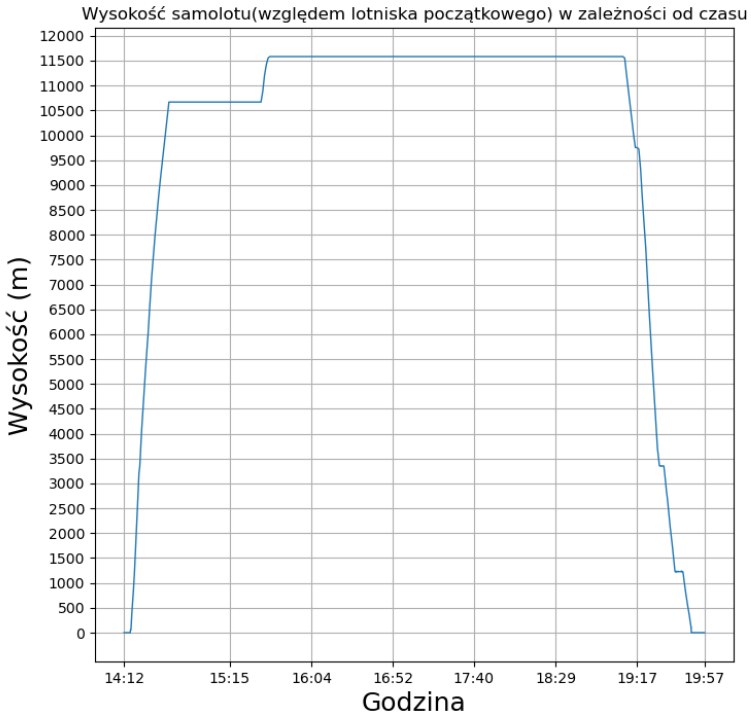


2. Wykres sky plot w układzie lotniska początkowego.

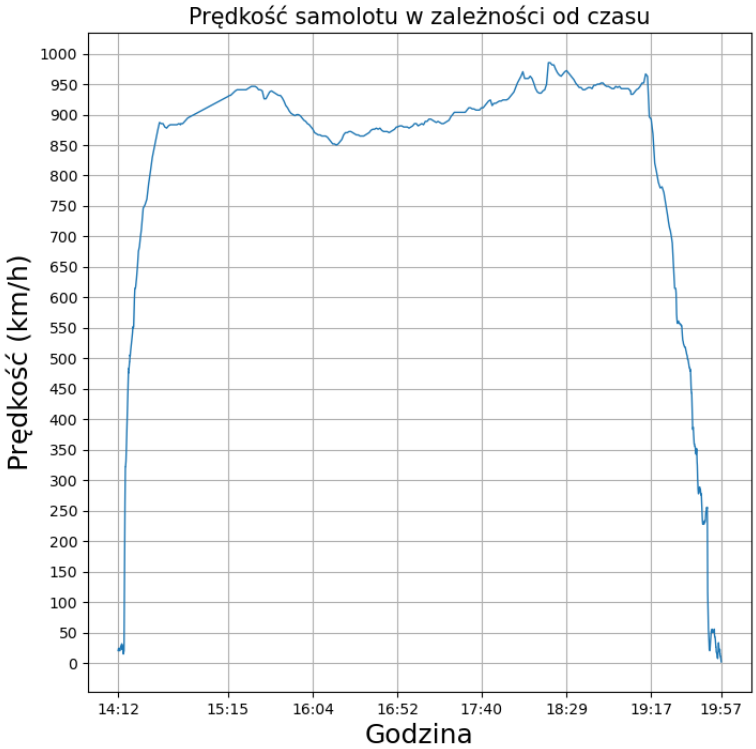


Samolot znika za horyzontem między 350, a 400 kilometrem lotu. Do tego czasu jego azymut względem lotniska początkowego ma wartość między 0, a 45 stopni.

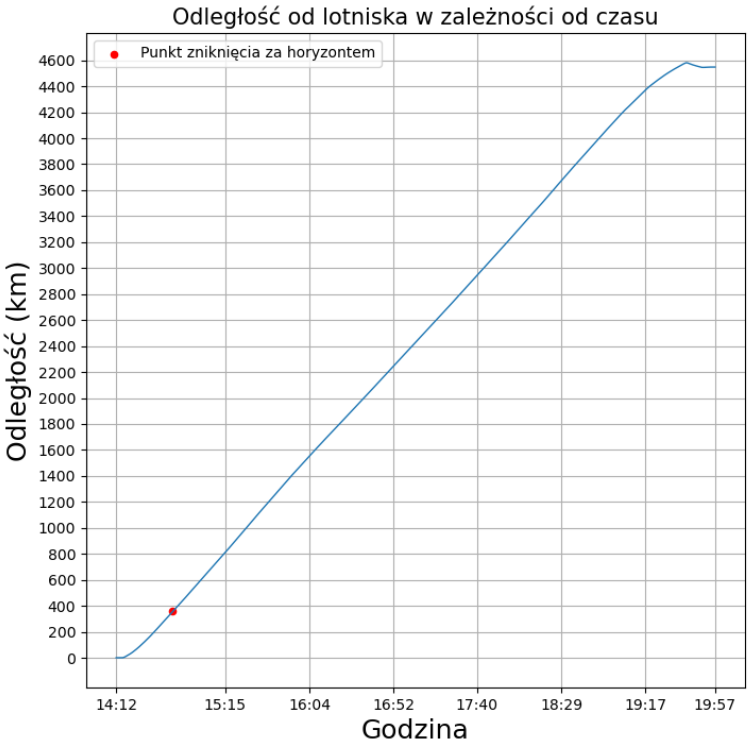
3. Wykres wysokości względem lotniska początkowego.



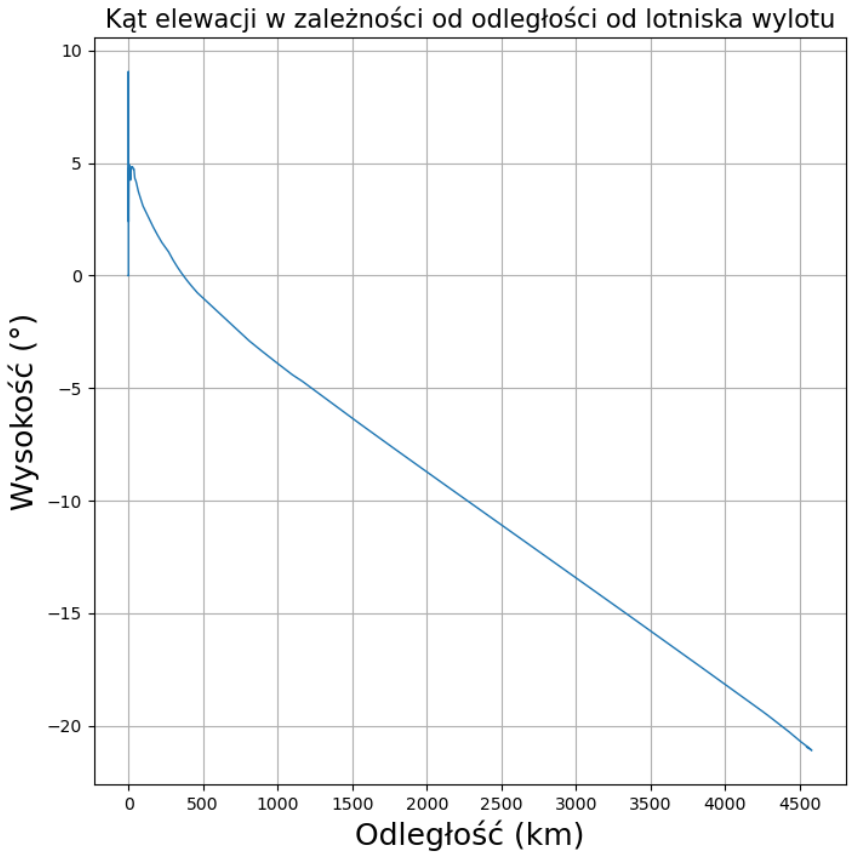
4. Wykres prędkości samolotu



5. Wykres odległości od lotniska początkowego.



6. Wykres kąta elewacji względem lotniska początkowego.



Z wizualizacji wynika, że lot trwał 5 godzin i 9 minut, samolot, tylko w początkowej fazie lotu znajdował się nad horyzontem, pod płaszczyzną horyzontu znalazł się już na około 390 kilometrze lotu około godziny 14:42. Cała długość trasy jaką pokonał to około 4600 kilometrów. Przez większość lotu utrzymywał maksymalną wysokość (11500m), wznoszenie zakończył na około 300 kilometrze trasy, zaś obniżanie lotu rozpoczął na około 300 kilometrów przed lotniskiem docelowym. Maksymalna prędkość wynosiła około 980km/h, zaś minimalna prędkość samolotu po osiągnięciu wysokości przelotowej wyniosła 850 km\h. Wraz z obniżaniem lotu przed lotniskiem docelowym prędkość samolotu malała. Maksymalny kąt elewacji wyniósł około 8 stopni, wraz z oddalaniem się od lotniska wartość kąta elewacji gwałtownie spadła, a po wylądowaniu we Frankfurcie kąt elewacji wyniósł około -21 stopni.